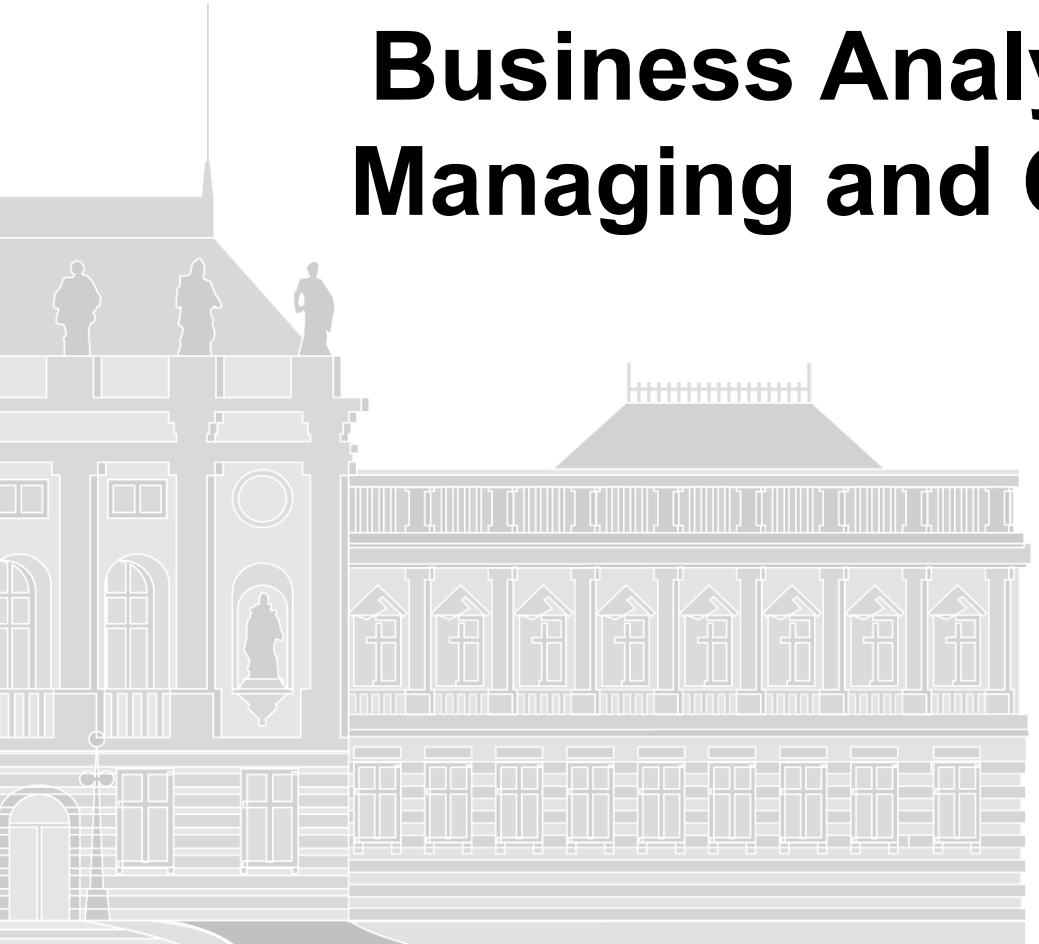
Business Analytics with R – Managing and Cleaning Data



Outline

- Goals of this lecture
- Base Functions vs External Packages
- Data Management with tidyverse
- Data Cleaning

Goals of the current lecture

- Students can clean and prepare data

Base Functions vs External Packages

Base Functions

- The basic R installation package comes with a handful of functions already installed.
- Those built-in functions are functions that you will encounter frequently when using R.
- We already used such base functions, e.g. mean()

Functions from external packages

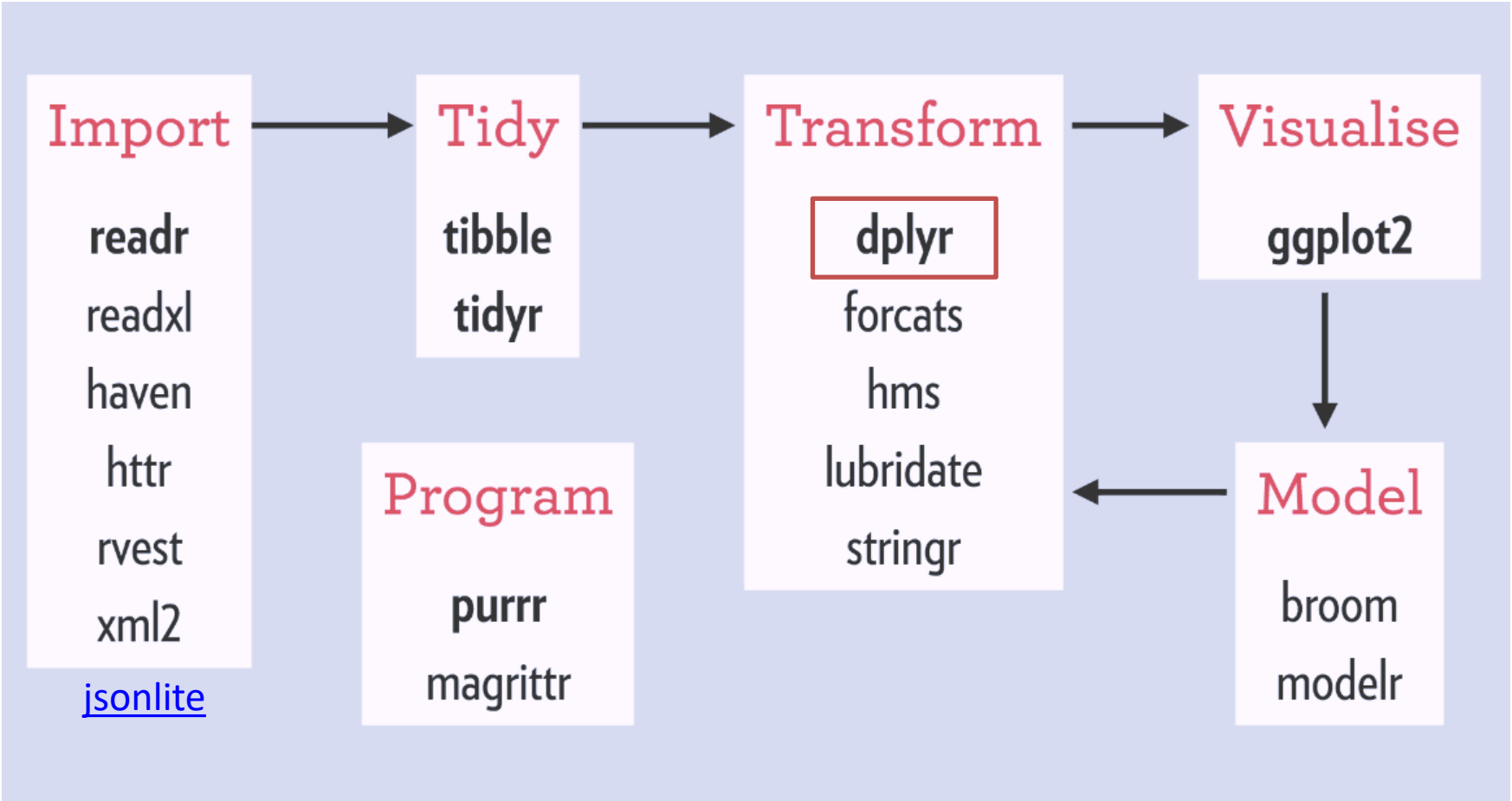
- You have the option to add more functions from external sources by installing external "packages". Those packages are (usually) available free of charge.
  - A list of all certified packages can be found on the R website (https://cran.r-project.org/web/packages/index.html).
- Anyone can write an R package and publish it. Most external packages are written by university employees or practitioners from a variety of industries.
- External packages cover a wide range of topics
  - Machine Learning, Advanced statistical analysis
  - Economic forecasting, Financial analysis, or
  - Data management

Data Management with tidyverse

The Tidyverse

- The tidyverse is a coherent system of packages for data manipulation, exploration and visualization that share a common design philosophy.
  - consistent functions,
  - workflow coverage,
  - a parsimonious approach to the development of data science tools, and
  - the possibility of greater productivity.
- install.packages(“tidyverse”)

The Tidyverse



The dplyr-package

- What is dplyr?
  - dplyr is a new package which provides a set of tools for efficiently manipulating datasets in R
- Why dplyr?
  - **Speed.** dplyr is faster than many alternatives.
  - **Flexibility.** dplyr can be used to manage data stored in many different sources (i.e. Excel- and CSV files, SQL databases,…)
  - **Cleaner Code.** dplyr makes it easy to write clean code.
  - **Simpler Code.** The code is easy to write and to follow.

Pipes

- What is a pipe?
  - Pipes let you pass an intermediate result onto the next function.
  - Real Estate Agent:
    - Filter all houses with 2 bedrooms and compute the average price of the home.
    - Real_estate_data %>%

```
filter(bedrooms==2) %>%
summarize(mean.prize = mean(prize))
```

Pipes

- Why use pipes?
  - You'll structure the sequence of your data operations from left to right, as apposed to from inside and out;
  - You'll avoid nested function calls;
  - You'll minimize the need for local variables and function definitions; and
  - You'll make it easy to add steps anywhere in the sequence of operations.

➔ Your code becomes easier to read, debug and less prone to errors

Pipes

- How can I use pipes?
  - "%>%" operator passes the outputs of one function as an input into the next function in the pipe.

```
# Initialize `x`
> x <- c(0.109, 0.359, 0.63, 0.996, 0.515, 0.142, 0.017, 0.829, 0.907)

# Compute the logarithm of `x`, return suitably lagged and iterated differences,
# compute the exponential function and round the result
> round(exp(diff(log(x))), 1)

# Import `magrittr`
> library(magrittr)

# Perform the same computations on `x` as above
> x %>%
>     log() %>%
>     diff() %>%
>     exp() %>%
>     round(1)
```

dplyr::filter

- Sometimes you only want to look at observations with certain characteristics (i.e. homes that were newly constructed, or homes with electric heating systems).
- You can filter these observations with the command „filter(conditions)“

```
# Import data from housing dataset
housing_data <- read.csv('Housing_Prices.csv',header=TRUE,sep=';')

# Import `dplyr`
library(dplyr)

# Filter new homes with electric heating and store them in a new dataset
housing_data_electric <- housing_data %>%
  filter(Fuel.Type=="Electric"))

#Looking at the new dataset
print(housing_data_electric)
```

Operator	Description
<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to
==	exactly equal to
!=	not equal to
x | y	x OR y
x & y	x AND y

dplyr::select

- You can also remove certain variables/columns from a dataframe using the command „select“

```
# Import data from housing dataset
housing_data <- read.csv('Housing_Prices.csv',header=TRUE,sep=';')

# Import `dplyr`
library(dplyr)

# Remove all attributes except for Price and Age
housing_data_reduced <- housing_data %>%
    select(Price, Age)

#Looking at the new dataset
print(housing_data_reduced)
```

dplyr::mutate

- mutate() computes a new variables

```
# Import data from housing dataset
housing_data <- read.csv('Housing_Prices.csv',header=TRUE,sep=';')

# Import `dplyr`
library(dplyr)

# Calculate price per square metre
housing_data <- housing_data %>% #store all changes below in the same dataset!
  mutate(price_sqrft = Price / Living.Area)

print(housing_data)
```

Data Cleaning

Messy dataset

```
> head(df)
          regio1 serviceCharge                    heatingType telekomTvOffer      cId newlyConst balcony
1 Sachsen_Anhalt           120                central_heating  ONE_YEAR_FREE   796687          n       y
2        Sachsen           390                central_heating  ONE_YEAR_FREE   226664          n       y
3         Hessen           380                    gas_heating  ONE_YEAR_FREE 17271157          n       y
4         Berlin           200  combined_heat_and_power_plant  ONE_YEAR_FREE 17720196          n       y
5        Sachsen           150                central_heating  ONE_YEAR_FREE 16482623          n       n
6         Hessen           110 self_contained_central_heating  ONE_YEAR_FREE 15462469          n       n

                              picture
1 https://pictures.immobilienscout24.de/listings/4b32bd2b-2c54-43f8-ace0-55bbfe3008dd-1244703191.jpg/ORIG/resize/118x118%3E/e
xtent/118x118/format/jpg/quality/80
2 https://pictures.immobilienscout24.de/listings/784d761e-d9ef-40c3-9e69-cb608dc8c306-1286019896.jpg/ORIG/resize/118x118%3E/e
xtent/118x118/format/jpg/quality/80
3 https://pictures.immobilienscout24.de/listings/41dfe454-4967-4882-828f-07ece260379f-1271813924.jpg/ORIG/resize/118x118%3E/e
xtent/118x118/format/jpg/quality/80
4 https://pictures.immobilienscout24.de/listings/f69945b8-7fde-43e4-bed5-f11d8e6852ab-1284406856.jpg/ORIG/resize/118x118%3E/e
xtent/118x118/format/jpg/quality/80
5 https://pictures.immobilienscout24.de/listings/b3a5edc3-7e08-4e68-9da0-4f3ad419e9c7-1272777475.jpg/ORIG/resize/118x118%3E/e
xtent/118x118/format/jpg/quality/80
6 https://pictures.immobilienscout24.de/listings/cfe7d13c-cb4e-4498-b899-cbd26884ef36-1203628714.jpg/ORIG/resize/118x118%3E/e
xtent/118x118/format/jpg/quality/80
  picturecount pricetrend telekomUploadSpeed totalRent    telekomTrackingGroup telekomInternetTechnology yearConstructed
1            8       3.33          40 MBit/s       440 telekom_layer_magenta_l                Ä¼ber VDSL            1903
2            8       2.35          40 MBit/s      1550 telekom_layer_magenta_l                Ä¼ber VDSL            1900
3           10       5.10          10 MBit/s      3130 telekom_layer_magenta_m                Ä¼ber VDSL            1900
4           11       7.23          40 MBit/s       920 telekom_layer_magenta_l           Ä¼ber Glasfaser            2017
5           11       6.45          40 MBit/s       600 telekom_layer_magenta_l                Ä¼ber VDSL            2008
6            4       3.85          10 MBit/s       310 telekom_layer_magenta_m                Ä¼ber VDSL            1906
  telekomInternetType ga_cd_test_cxp_ads pricetrendbuy   scoutId    firingTypes hasKitchen ExclusiveExpose
1             REGULAR         ADS1406_ON         11.24 107685379           <NA>          n           FALSE
2             REGULAR         ADS1406_ON          8.66 110611624            gas          n           FALSE
3             REGULAR         ADS1406_ON         13.57 109807432            gas          n           FALSE
4             REGULAR         ADS1406_ON         12.76 110710462            gas          y           FALSE
5             REGULAR         ADS1406_ON          <NA>  78637694 no_information          n           FALSE
6             REGULAR         ADS1406_ON          <NA>  94603873            gas          n           FALSE
  telekomInternetProductName        geo_bln cellar yearConstructedRange baseRent    houseNumber         energyType
1           MagentaZuhause L Sachsen_Anhalt      n                    1      320             11 energy_consumption
2           MagentaZuhause L        Sachsen      n                    1     1160 no_information energy_consumption
3           MagentaZuhause M         Hessen      y                    1     2750             44               <NA>
4           MagentaZuhause L         Berlin      y                    9      920             36               <NA>
5           MagentaZuhause L        Sachsen      y                    7      450             71               <NA>
6           MagentaZuhause M         Hessen      n                    1      200              9               <NA>
```

Messy dataset

Douplicate observations

```
> head(df)
          regio1 serviceCharge                    heatingType telekomTvOffer       cId newlyConst balcony
1 Sachsen_Anhalt           120                central_heating  ONE_YEAR_FREE    796687          n       y
2        Sachsen           390                central_heating  ONE_YEAR_FREE    226664          n       y
3         Hessen           380                    gas_heating  ONE_YEAR_FREE 17271157          n       y
4         Berlin           200  combined_heat_and_power_plant  ONE_YEAR_FREE 17720196          n       y
5        Sachsen           150                central_heating  ONE_YEAR_FREE 16482623          n       n
6         Hessen           110 self_contained_central_heating  ONE_YEAR_FREE 15462469          n       n

                                picture
1 https://pictures.immobilienscout24.de/listings/4b32bd2b-2c54-43f8-ace0-55bbfe3008dd-1244703191.jpg/ORIG/resize/118x118%3E/e
xtent/118x118/format/jpg/quality/80
2 https://pictures.immobilienscout24.de/listings/784d761e-d9ef-40c3-9e69-cb608dc8c306-1286019896.jpg/ORIG/resize/118x118%3E/e
xtent/118x118/format/jpg/quality/80
3 https://pictures.immobilienscout24.de/listings/41dfe454-4967-4882-828f-07ece260379f-1271813924.jpg/ORIG/resize/118x118%3E/e
xtent/118x118/format/jpg/quality/80
4 https://pictures.immobilienscout24.de/listings/f69945b8-7fde-43e4-bed5-f11d8e6852ab-1284406856.jpg/ORIG/resize/118x118%3E/e
xtent/118x118/format/jpg/quality/80
5 https://pictures.immobilienscout24.de/listings/[illegible]da0-4f3ad419e9c7-1272777475.jpg/ORIG/resize/118x118%3E/e
xtent/118x118/format/jpg/quality/80
6 https://pictures.immobilienscout24.de/l[illegible]899-cbd26884ef36-1203628714.jpg/ORIG/resize/118x118%3E/e
xtent/118x118/format/jpg/quality/80
  picturecount pricetrend telekomUploadspe[illegible]kingGroup telekomInternetTechnology yearConstructed
1            8       3.33          40 MBit/[illegible] [illegible]magenta_l                Ä¼ber VDSL            1903
2            8       2.35          40 MBit/s      1550 telekom_layer_magenta_l                Ä¼ber VDSL            1900
3           10       5.10          10 MBit/s      3130 telekom_layer_magenta_m                Ä¼ber VDSL            1900
4           11       7.23          40 MBit/s       920 telekom_layer_magenta_l           Ä¼ber Glasfaser            2017
5           11       6.45          40 MBit/s       600 telekom_layer_magenta_l                Ä¼ber VDSL            2008
6            4       3.85          10 MBit/s       310 telekom_layer_magenta_m                Ä¼ber VDSL            1906
  telekomInternetType ga_cd test_cxp_ads pricetrendbuy   scoutId    firingTypes hasKitchen ExclusiveExpose
1             REGULAR         ADS1406_ON         11.24 107685379           <NA>          n           FALSE
2             REGULAR         ADS1406_ON          8.66 110611624            gas          n           FALSE
3             REGULAR         ADS1406_ON         13.57 109807432            gas          n           FALSE
4             REGULAR         ADS1406_ON         12.76 110710462            gas          y           FALSE
5             REGULAR         ADS1406_ON          <NA>  78637694 no_information          n           FALSE
6             REGULAR         ADS1406_ON          <NA>  94603873            gas          n           FALSE
  telekomInternetProductName        geo_bln cellar yearConstructedRange baseRent    houseNumber         energyType
1           MagentaZuhause L Sachsen_Anhalt      n                    1      320             11 energy_consumption
2           MagentaZuhause L        Sachsen      n                    1     1160 no_information energy_consumption
3           MagentaZuhause M         Hessen      y                    1     2750             44               <NA>
4           MagentaZuhause L         Berlin      y                    9      920             36               <NA>
5           MagentaZuhause L        Sachsen      y                    7      450             71               <NA>
6           MagentaZuhause M         Hessen      n                    1      200              9               <NA>
```

Variables that are not defined well

Missing values

Removing Redundant Attributes

- Once you have selected a set of attributes that you want to keep, you can use the „select()“ command from the tidyverse package to select the attributes you want to keep.
- <u>If 10% (or more) of the observations in a variable are missing, remove the variable.</u>

```
> library(tidyverse)
> df <- read.csv("Dataset_Teachers.csv")
> head(df)
> summary(df)

> df.1 <- df %>%
    select(regio1,
       balcony,
       cId,
       regio2,
       totalRent,
       yearConstructed,
       noRooms,
       cellar,
       hasKitchen)
```

Removing Redundant Observations

- Duplicated observations can be easily removed using the command „unique(*object*)“. When used on a dataframe, the unique command removes all observations that are identical.
- In some occasions, it is possible that you have several observations from the same entity/person. Even though this may sometimes wanted, you may want to only a single observation of an individual in others. In these scenarios, the command “unique()” can be used as well.
- <u>If 5% (or less) of the observations in a variable are missing, remove the observations.</u>

```
> df.1 <- unique(df.1)

> Ids.unique <- unique(df.1$cId)
…
```

Dealing with missing values

- R stores missing values as *NA*.

1. Detecting missing values.
    1. Function "is.na(**object name**)"
        1. **Input**: a data frame, vector or matrix
        2. **Output**: Categorical data frame, vector or matrix (True if the corresponding element in the object is missing and False, if the corresponding element in the object is not missing)
2. Removing missing values.
3. Estimating missing values

```
> is.na(df.1)
> colSums(!(is.na(df.1)))     # Counts how many non-missing entries there are in each column
> colSums((is.na(df.1)))      # Counts how many values are missing in each one of the attributes.
```

Dealing with missing values

```
> is.na(df.1)
      regio1 balcony regio2 totalRent yearConstructed noRooms cellar hasKitchen
 [1,]  FALSE   FALSE  FALSE     FALSE           FALSE   FALSE  FALSE      FALSE
 [2,]  FALSE   FALSE  FALSE     FALSE           FALSE   FALSE  FALSE      FALSE
 [3,]  FALSE   FALSE  FALSE     FALSE           FALSE   FALSE  FALSE      FALSE
 [4,]  FALSE   FALSE  FALSE     FALSE           FALSE   FALSE  FALSE      FALSE
 [5,]  FALSE   FALSE  FALSE     FALSE           FALSE   FALSE  FALSE      FALSE
 [6,]  FALSE   FALSE  FALSE     FALSE           FALSE   FALSE  FALSE      FALSE
 [7,]  FALSE   FALSE  FALSE     FALSE           FALSE   FALSE  FALSE      FALSE
 [8,]  FALSE   FALSE  FALSE     FALSE           FALSE   FALSE  FALSE      FALSE
 [9,]  FALSE   FALSE  FALSE     FALSE           FALSE   FALSE  FALSE      FALSE
[10,]  FALSE   FALSE  FALSE     FALSE           FALSE   FALSE  FALSE      FALSE
[11,]  FALSE   FALSE  FALSE     FALSE            TRUE   FALSE  FALSE      FALSE
[12,]  FALSE   FALSE  FALSE     FALSE            TRUE   FALSE  FALSE      FALSE
[13,]  FALSE   FALSE  FALSE     FALSE            TRUE   FALSE  FALSE      FALSE
[14,]  FALSE   FALSE  FALSE     FALSE            TRUE   FALSE  FALSE      FALSE
[15,]  FALSE   FALSE  FALSE     FALSE           FALSE   FALSE  FALSE      FALSE
[16,]  FALSE   FALSE  FALSE     FALSE            TRUE   FALSE  FALSE      FALSE
[17,]  FALSE   FALSE  FALSE     FALSE           FALSE   FALSE  FALSE      FALSE
[18,]  FALSE   FALSE  FALSE     FALSE           FALSE   FALSE  FALSE      FALSE
[19,]  FALSE   FALSE  FALSE     FALSE           FALSE   FALSE  FALSE      FALSE
[20,]  FALSE   FALSE  FALSE     FALSE           FALSE   FALSE  FALSE      FALSE
[21,]  FALSE   FALSE  FALSE     FALSE           FALSE   FALSE  FALSE      FALSE
[22,]  FALSE   FALSE  FALSE     FALSE           FALSE   FALSE  FALSE      FALSE
[23,]  FALSE   FALSE  FALSE     FALSE           FALSE   FALSE  FALSE      FALSE
[24,]  FALSE   FALSE  FALSE     FALSE            TRUE   FALSE  FALSE      FALSE
[25,]  FALSE   FALSE  FALSE     FALSE           FALSE   FALSE  FALSE      FALSE
[26,]  FALSE   FALSE  FALSE     FALSE           FALSE   FALSE  FALSE      FALSE
[27,]  FALSE   FALSE  FALSE     FALSE           FALSE   FALSE  FALSE      FALSE
[28,]  FALSE   FALSE  FALSE     FALSE           FALSE   FALSE  FALSE      FALSE
[29,]  FALSE   FALSE  FALSE     FALSE           FALSE   FALSE  FALSE      FALSE
[30,]  FALSE   FALSE  FALSE     FALSE            TRUE   FALSE  FALSE      FALSE
[31,]  FALSE   FALSE  FALSE     FALSE           FALSE   FALSE  FALSE      FALSE
[32,]  FALSE   FALSE  FALSE     FALSE            TRUE   FALSE  FALSE      FALSE
[33,]  FALSE   FALSE  FALSE     FALSE            TRUE   FALSE  FALSE      FALSE
[34,]  FALSE   FALSE  FALSE     FALSE           FALSE   FALSE  FALSE      FALSE
[35,]  FALSE   FALSE  FALSE     FALSE           FALSE   FALSE  FALSE      FALSE
[36,]  FALSE   FALSE  FALSE     FALSE           FALSE   FALSE  FALSE      FALSE
[37,]  FALSE   FALSE  FALSE     FALSE           FALSE   FALSE  FALSE      FALSE
[38,]  FALSE   FALSE  FALSE     FALSE            TRUE   FALSE  FALSE      FALSE
[39,]  FALSE   FALSE  FALSE     FALSE           FALSE   FALSE  FALSE      FALSE
[40,]  FALSE   FALSE  FALSE     FALSE            TRUE   FALSE  FALSE      FALSE
```

Dealing with missing values

- R stores missing values as *NA*.

1. Detecting missing values.
2. Removing missing values.
    1. The function “complete()” returns only complete rows of a dataframe.
    2. You can also remove all observations that do not have an entry in a certain attribute.
3. Estimating missing values

```
> is.na(df.1)
> df.1 <- na.omit(df.1)
> df.1 <- df.1 %>%
    filter(!is.na(df.1$yearConstructed))
```

Dealing with missing values

- R stores missing values as *NA*.

1. Detecting missing values.
2. Removing missing values.
3. Estimating missing values
   1. We have not yet covered the most common estimation methods (regression and classification). We will show you how to implement them at a later point.
   2. Mean or Median.

- <u>If between 5% and 10% of the observations in a variable are missing, you will need to think about whether you really need the variable.</u>

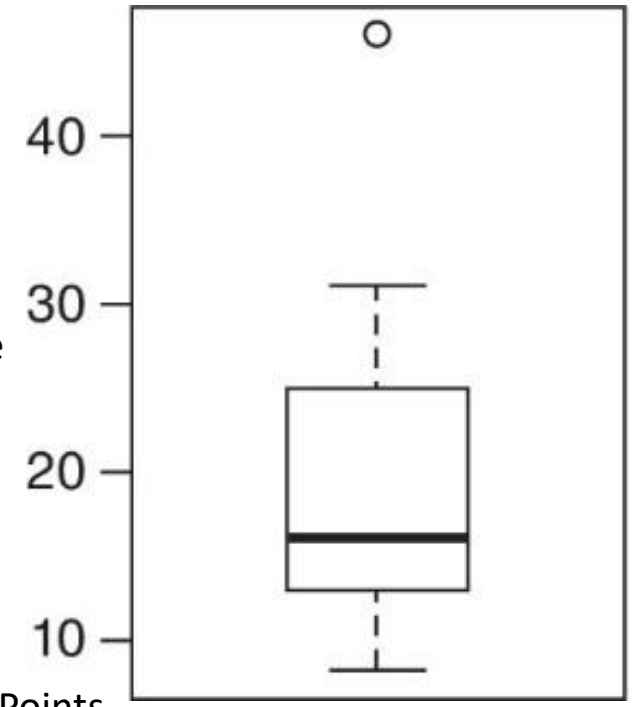
Dealing with Outliers

- Outliers are observations showing values outside the expected range, or data that is substantial different from the rest

1. “Mechanical” approaches to identify outliers
   1. Observations outside e.g. 1.5*InterquartileRange
   2. Observations outside e.g. 3*Standard Deviation
   3. Visualization can be used to detect unusual observations
   4. And many others, like finite mixture models etc.
2. You should not, however, just automatically remove those cases.
   1. Remember the normal distribution: 99.74% of cases should lie within 3*standard deviations
   2. If you have 70.000 observations, you would expect around 182 “outliers”, that are in fact perfectly good data
3. Outliers can be produced by different causes
   1. Bad data / errors
   2. True outliers, like the heaviest baby ever born with 10.2kg (average in Austria 2017: 3.317kg; 0.9% heavier than 4.5kg)

Dealing with Outliers

1. Detecting Outliers – Option 1
   1. Compute first and third quartile, Q_1 and Q_3
   2. Compute interquartile range (IQR)
   3. Compute boundary points
   4. Check which observations have values outside the boundaries.
2. Removing Outliers
3. Truncating Outliers

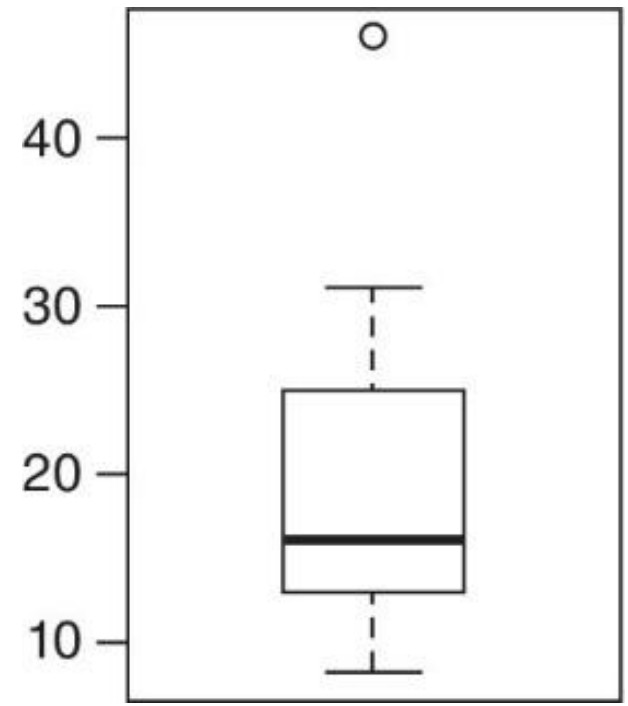



```
# Computing quartiles
> quartiles <- quantile(df.1$totalRent)
> Q.1 <- quartiles[2]
> Q.3 <- quartiles[4]

# Computing Inter Quartile Range
> inter.quartile.range <- IQR(df.1$totalRent)
```

```
# Computing Boundary Points
> lower.bound <- Q.1-1.5*inter.quartile.range
> upper.bound <- Q.3+1.5*inter.quartile.range

# Check for outliers
> df.1.outliers.below <- df.1$totalRent < lower.bound
> df.1.outliers.above <- df.1$totalRent > upper.bound
```

Dealing with Outliers

1. Detecting Outliers – Option 2
   1. Normalize your dataset by computing z-scores.
   2. Compute mean/median of the attribute.
   3. Compute standard deviation of the attribute
   4. Compute boundary points (mean/median ±3 SD)
   5. Check which observations have values outside the boundaries.
2. Removing Outliers
3. Truncating Outliers




```
# Normalize Dataset by computing z-scores
df.1$scaledRent <- scale(df.1$totalRent)

# Computing mean and standard deviation
> mean.rent <- mean(df.1$scaledRent)
> sd.rent <- sd(df.1$scaledRent)
```

```
# Computing Boundary Points
> lower.bound <- rent – 3 * sd.rent
> upper.bound <- rent + 3 * sd.rent

# Check for outliers
> df.1.outliers.below <- df.1$totalRent < lower.bound
> df.1.outliers.above <- df.1$totalRent > upper.bound
```

Dealing with Outliers

- Outliers are easy to remove, but all exclusion of data can change your results and should be done with care.
- You should investigate the cases you exclude (with the method depending on the number of potential outliers).
- You should check the robustness of your results with regard to removal of outliers.

1. Detecting Outliers
2. Removing Outliers
   1. Removing outliers can be done by using the “filter()” function from the dplyr package.
3. Truncating Outliers

```
# Removing outliers
> df.1 <- df.1 %>%
  filter(!(df.1$totalRent > upper.bound)) %>%
  filter(!(df.1$totalRent < lower.bound))
```

Dealing with Outliers

- Luckily, outliers are fairly easy to truncate as well.

1. Detecting Outliers
2. Removing Outliers
3. Truncating Outliers
    1. Truncating outliers means that we replace the observed variable values with the upper/lower bound found earlier.

```
# Truncating Outliers
> df.1$totalRent[(df.1$totalRent < lower.bound)] <- lower.bound
> df.1$totalRent[(df.1$totalRent > upper.bound)] <- upper.bound
```

Questions?